

Humanoid Standup from Arbitrary Pose

Humanoid Standup from Arbitrary Pose

Task #8 - A humanoid robot lying on the ground in a random initial pose (face-up, face-down, or on-side) must learn to stand up and balance. A single controller handles all starting orientations.

1. Problem

Humanoid getup is the hard sibling of humanoid standing-balance. Balance is local: linearize about the upright equilibrium, find an LQR / PD gain, done. Getup is global: the body has to traverse a long sequence of mechanically distinct phases (lying, sitting, kneeling, partial squat, standing), most of which are far from any equilibrium of the unforced dynamics, and which require qualitatively different motion primitives (sit-up from supine, push-up from prone, side-roll, leg extension). Reward-wise the task is also sparse: the obvious “head height” or “torso upright” signals are zero or near-zero over most of the configuration space the policy needs to traverse, and the only really “good” state is a thin slice of fully-standing posture.

The take-home spec adds two constraints that make it harder than single-pose getup demos in the literature:

- **Multiple init orientations under one policy.** The same controller has to recover from face-up, face-down, on-side. These require different first moves (push-up vs sit-up vs roll); a tracking-based approach would need a *library* of references.
- **Stable standing balance, not transient.** The policy must internally stabilize the upright pose (no unforced equilibrium under zero ctrl; 8 unstable modes in the local linearization at the policy’s operating point, see §5) rather than just touch the upright zone and fall back.

1a. Assumptions (scope)

- **5 anchor poses cover the spec’s “random initial pose.”** Uniform mix over {standing, supine, prone, side, kneel} plus ± 0.02 rad joint jitter (probed up to ± 0.3 rad + random yaw + 10 cm xy offset). Not a continuous distribution over arbitrary configurations - a randomly- twisted limbs-tangled start isn’t in-distribution.
- **No physics-parameter DR.** Mass, inertia, friction, motor torque limits held at dm_control defaults. Motor-model DR is added in §5 as a separate evaluation. Lifted by standard DR if sim-to-real matters.

- **dm_control humanoid, not a real robot.** Fixed torque actuators, no per-joint PD or motor parameters to co-tune. Existing real-robot getup work (HumanUP, RSS 2025) jointly tunes motor parameters with the policy for exactly this reason.
-

1b. Prior approaches (and why they were abandoned)

Before settling on the brax PPO + mujoco_playground setup used here, four other approaches were tried on a CPU MuJoCo + multiprocessing prototype. None converged on a usable controller within a reasonable wall-clock; each plateaued on a different *weird* local minimum that the reward shape permitted. The common bottleneck was env throughput, which is what motivated the switch to MJX + GPU below.

(a) PPO from scratch (single-pose supine)

A from-scratch PPO actor-critic on a single supine init, with a dense head-height reward and no reference.

- **Local minimum found:** “standing on head” - the policy discovered it could *maximize torso z* by inverting itself onto its head/shoulders, legs sticking up. Head reward fired hard; standing reward stayed at zero; the body never touched its feet to the floor.
- **Why:** the dense reward rewarded “torso high” without distinguishing “torso high on feet” from “torso high on head.”
- **Bottleneck:** the multiprocessing vector env topped out at ~100 concurrent CPU MuJoCo envs. PPO sample efficiency at that scale meant hours per real reward jump; full standing would have taken days even if the reward shape had been right.

(b) MPPI offline planning

Pure predictive sampling from the supine keyframe, stitched in two stages: a **rise** phase (MPPI with a height + upright cost driving the torso up) followed by a **settle** phase (MPPI with a state-tracking cost to bring $|q_{vel}|$ to zero at standing - needed because the rise phase arrived at standing *with momentum* and otherwise fell straight through it).

- **Local minimum found:** the rise-only planner consistently produced a “leap-through” trajectory - peak torso z at ~1.25 m, then collapse, because reaching standing was cheap but *staying* there required a separate velocity-damping cost the rise phase didn’t have. The two-stage stitch was the fix.
- **Strength:** no training; demonstrates the dynamics support the motion under bounded torque.
- **Weakness:** one trajectory, one init condition. Open-loop ctrl replay diverges in seconds under contact noise; closed-loop replanning every step is too slow for real-time.

The cleaned port of this MPC ships here as `cp-mpc` (cost in Appendix B).

(c) MPPI + learned residual

A residual policy: action $\mathbf{a}_t$, where $\mathbf{a}_t^{\text{open-loop}}$ is the open-loop ctrl from (b)'s saved trajectory and $\mathbf{r}_t$ is a small MLP trained with PPO to add corrections. Reward combined tracking the MPPI joint trajectory with the standing terms.

- **Local minimum found:** the residual collapsed to ~zero output. The MPPI feed-forward was strong enough to mostly succeed and the residual couldn't *improve* tracking error without breaking the timing alignment, so PPO learned the safest policy: do nothing extra. The controller's behavior was just the MPPI trajectory, with all its fragility.
- **Why:** residual policies need the base controller to be *bad enough* that corrections clearly help. A near-perfect MPPI baseline removes the gradient.

(d) PPO tracking the MPPI trajectory (no residual)

Drop the residual idea, train PPO from scratch with a dense reward that *tracks* the MPPI joint trajectory: $\mathbf{r}_t = \mathbf{a}_t - \mathbf{a}_t^{\text{ref}}$ plus standing reward at the end. Phase-randomized reset so the agent learns to recover from any point along the reference.

- **Local minimum found:** “crouch with hands on floor.” The policy discovered that mirroring the MPPI joint angles approximately while keeping its hands on the ground for support let it score most of the tracking reward without committing to the actual standup - it shadowed the upper-body motion in a crouched, hand-supported pose.
- **MJX port:** porting this env to MJX (vmap-batched on a single H100, Newton solver - CG diverged from the MPPI reference's contact-rich state distribution) was the first step out of the CPU env-throughput wall. It scaled to 2048 envs and trained at ~2M env-steps/sec.
- **Why it isn't the deliverable:** the tracking reward locks the policy to one reference trajectory shape. For the take-home spec (single controller across face-up / face-down / on-side) a tracking policy would need a *library* of reference trajectories - or just learn the standup behavior end-to-end without tracking, which is what this repo does.

The pivot

- a. and (b) suffered from reward shapes that admitted weird minima;
 - b. and (d) from baseline-reference dependence and CPU env throughput. The fix that mattered most for wall-clock was moving the simulator to MJX so a single H100 could replace ~64 CPU env workers, which made iteration on reward shape *feasible in a human-scale debug loop*. The rest of this writeup is what that loop produced.
-

2. Approach

Stack: brax PPO (Google) + mujoco_playground dm_control humanoid (DeepMind), trained on Modal H100. Per-run compute ranged from ~90M env-steps (stock baseline) to 1B (v9 with DR); the v8-long deliverable was a ~500M-target run that brax overran to 530M.

Robot: 21-DOF dm_control humanoid (DeepMind’s standard model: torque actuators on each hinge, freejoint root, no per-joint PD or motor-parameter tuning to optimize over). Existing humanoid-standup work in the literature (notably HumanUP, RSS 2025) targets the Unitree G1 and jointly optimizes *both* a policy and motor / PD gains - a richer problem than this one, where the actuators are fixed and only the policy is learned. The simplification keeps the experiment focused on RL methodology rather than sim-to-real transfer.

Algorithm: brax PPO with mujoco_playground’s published hyperparameters for HumanoidStand (num_envs=2048, lr=1e-3, discounting=0.995, GAE- λ =0.95, ...). Network widened to [512, 256, 128] for both policy and value; brax’s default policy [32]*4 is ~50x smaller and turns out to under-fit whole-body coordination.

Env subclass (HumanoidGetUp): overrides reset() for a 5-way init mix {standing, supine, prone, side, kneel} and adds a dense exponential head-height bonus to the stock multiplicative reward.

3. Diagnostic Journey (v1 → v8)

Eight iterations. Each row notes the **single intentional change** and what the data showed.

v1 - supine init + naive reward

Stock reward identically 0 from supine (standing = $\text{tol}(\text{head_z}, (1.4, \text{inf}), 0.35)$ saturates below head_z=1.05). With zero advantages, PPO’s entropy term blew up actions until physics produced NaN qvel. **Fix:** add a dense signal.

v2 - added $\text{clip}(\text{head_z} / 1.4)$ bonus, weight 0.5

Reward hacking. Policy parked at head_z ≈ 0.77 m (sit/squat) where clip saturated. **Fix:** non-saturating exp shape.

v3 - exponential reward $\exp(-(z-1.4)^2/0.49)$ + 50/50 init mix

Real progress - stand 0 → 165, no hacking signature, all reward components climbing together. Final policy reached coordinated kneeling-with-hand-support. Two changes layered: exp shape (removes v2’s saturation flat) and 50/50 mix (50% standing / 50% supine inits — standard state-init trick from getup literature, lets the critic learn V(stand) directly and propagate via GAE).

Bug discovered later: the supine quat was [0.7071, 0, +0.7071, 0] (+90° about y, **face-down**), not supine. So v3 was actually “getup-from-PRONE.” Caught when the trained policy was viewed from a corrected supine init and just lay flat.

v4 - corrected supine pose (-y quat), everything else same as v3

Slower than v3 (supine is mechanically harder), but real getup behavior. - head_bonus: 390 | stand: 72 | total: 262 - Doubled stand reward in the last eval interval (43 → 72) - breakthrough out of an early plateau.

Hand-tuned supine keyframe in configs/env.yaml.

v5 - tightened standing margin (0.35 → 0.1, no credit below 1.3 m)

Pre-committed plan in case v4 plateaued. Modestly worse (stand 46 vs v4's 72, a 36% drop); viewer behavior unchanged - both pose policies sit at kneel-with-hand-support.

Lesson: the kneel plateau is a *critic value-gradient* problem, not a reward-shape one - tightening the threshold doesn't add gradient toward standing, it just removes the gradient toward kneel.

v6 - multi-orientation init + bigger network + 200M steps

Two diagnosed fixes layered: 1. **5-way init mix** {standing, supine, prone, side, kneel-snapshot}. The kneel-snapshot is captured from a v4 rollout at the kneeling-with-hand-support plateau. Initing there gives the critic direct visibility into kneel→stand transitions - targeted fix for v5's diagnosed problem. 2. **Networks [512, 256, 128]** for both policy and value. brax PPO defaults are ~50× smaller for the policy and turned out to be a real undersize. 3. 200M timesteps (vs 60-88M before).

Result: breakthrough. Final at 265M steps (brax overran the target): - total reward = 1076 (vs stock standing-balance baseline 810) - stand = 695/1000 (per-step avg 0.695 - standing ~70% of all episode steps, averaged across all 5 init types) - head_bonus = 889/1000, upright = 980, move = 961, small_control = 937

The kneel→stand gap was crossed somewhere in the 200-235M step window (stand jumped 189 → 500 → 695 over the last three evals).

The combined fix - putting the critic at the stuck pose so it sees forward transitions, plus restoring an adequately-sized policy network - closed the value-gradient gap exactly as diagnosed.

v7 - extend v6 to 500M steps (same recipe)

More compute, no env/reward changes. Trained through several oscillation phases (e.g. stand 165 at 236M, 47 at 295M, breakthrough to 820 at 530M). Final at 530M: stand = 820, total = 1177. Substantially higher peak than v6 (1076) but in viewer the policy *wobbles* while standing - reaches the standing zone but keeps one foot tippy-toe and sways. With only one seed we can't distinguish recipe-level wobble from a single-seed late-training artifact.

v8 - v7 recipe + foot_flat reward + stillness penalty (gated on head_z>1.3)

Single intentional change from v7: add two reward terms that fire only once the body is in the standing region ($\text{head_z} > 1.3$ m), targeting the v7 wobble specifically. Reward shape became: $\text{stock} + 0.5 \cdot \text{head_bonus} + 1[\text{h} > 1.3] \cdot (0.3 \cdot \text{foot_flat} - 1.0 \cdot \text{stillness})$.

v8 (200M, deliverable's predecessor): stand 587, foot_flat 228, eval std 65. Foot orientation ~2.3× better than v7 (228 vs 99), eval std much lower.

v8-long (500M target → 530M actual): oscillated between strong evals (e.g. 413M: total 1460, stand 868) and partial collapses (472M, 530M: total ~669, stand ~315). Final landed on a trough. The 413M peak checkpoint is preserved by per-eval checkpointing and is the deliverable (§4). Mean of the last three eval intervals (413M, 472M, 530M) is total ≈ 932 , std ≈ 370 - a wide late-training range, not a clean plateau. The 413M ckpt was chosen because the viewer rollout shows visibly stable flat-footed standing; the 472M and 530M trough ckpts show the body reaching standing then immediately falling.

4. Results

Single controller, five init types. Deliverable: v8-long @ 413M step checkpoint.

	Stock (balance only)	v6	v7 @530M	v8-long @413M
Init pose	Upright	5-way	5-way	5-way
Train steps	90M	265M	530M (also oscillating)	413M (oscillating, peak)
total reward / ep	810	1076	1177	1460 (peak)
stand / ep (max 1000)	~ 810	695	820	868 (peak)
foot_flat / ep	-	~ 41	~ 99	754 (7.5 $\times$ v7)
stillness penalty / ep	-	28	45	40
eval std	low	~ 30	143	97

Both v7 and v8-long oscillated in late training (see §3 v7 / v8); the column entries are the final eval value for v7 (530M) and the peak ckpt for v8-long (413M). v7’s 530M happens to land on the recovery side of its swing rather than a trough; a different seed might land on a trough and look much worse.

The v8 reward redesign (stillness penalty + foot-flat reward, both gated on head_z > 1.3) traded a small amount of standing time for **substantially better standing quality**: foot_flat is 7.5 $\times$ higher than v7 (feet flatter, not on tippy-toes), eval std is 32% lower (more reliable per episode), while peak stand reward still beats v7.

Three caveats on this table: - foot_flat and stillness columns for v6 and v7 are re-evaluations under the v8 metric (those terms were introduced in v8). Re-evals run the existing v6/v7 policies through eval_reward.py against the v8 reward shape; they’re directly comparable but the v6/v7 policies were not trained to optimize them. - The v8-long row is the **413M peak**, not the final eval. v8-long oscillated and final.pkl (530M) landed on a trough (~ 669 reward). Mean of the last three eval intervals (413M, 472M, 530M) is ≈ 932 , std ≈ 370 . We ship the peak ckpt explicitly because viewer rollouts at 413M are visibly stable standing while 472M/530M visibly fall. - “eval std” is per-episode std reported by brax PPO during evaluation (across the eval batch at that step), not std across late evals.

Qualitative behavior (observed via MuJoCo viewer rollouts): - Coordinated multi-stage getup motion. From supine: rolls slightly, sits up with arm push, brings legs under, pushes through kneeling, extends to stand. - Different sequence for prone (push-up to all-fours → kneel → stand) and for side (roll → supine → standard sequence) - one policy, three motions. - **Stable standing pose** with both feet flat on the floor. No tippy-toe. Minor sway to maintain balance but no falling.

Important note on checkpoint selection. v8-long oscillated between high-quality (413M, 1460 reward) and partial collapses (472M, 530M, ~669 reward). `final.pkl` lands on the trough. The deliverable is therefore the explicitly-saved peak: `getup-v8-long/ckpt_000412876800.pkl`. Per-eval checkpointing (added in `train.py` via `policy_params_fn`) is what makes this recoverable without re-running.

Generalization probe (out-of-distribution inits)

Trained jitter was ± 0.02 rad (a few degrees) with fixed quaternions per init type. To test how far the policy generalizes, ran `cp-probe` (`src/controls_playground/scripts/probe.py`) - same 5 base poses but with **15× wider joint jitter** (± 0.3 rad), **full random yaw** ($\pm \pi$), and **± 10 cm xy offset**:

ep 0	PRONE	peak head_z	1.49	STOOD
ep 1	STANDING	peak head_z	1.72	STOOD
ep 2	STANDING	peak head_z	1.65	STOOD
ep 3	SIDE	peak head_z	1.49	STOOD
ep 4	SIDE	peak head_z	1.50	STOOD
ep 5	SIDE	peak head_z	1.50	STOOD
ep 6	KNEEL	peak head_z	1.48	STOOD
ep 7	KNEEL	peak head_z	1.50	STOOD
ep 8	SUPINE	peak head_z	1.50	STOOD
ep 9	STANDING	peak head_z	1.73	STOOD

10/10 stood. With $n=10$ this is a suggestive result, not a statistically tight one: the 95% Wilson interval is roughly 0.72-1.00, so all we can say is “success rate is plausibly above ~70%.” The pattern is at least consistent with the policy generalizing beyond the trained init distribution (the `dm_control` humanoid obs is naturally yaw-invariant and the multi-pose mix implicitly covers some joint variation), but a proper validation would need n in the hundreds at each perturbation level.

5. Controls extension

Two controls-flavored extensions on top of the trained RL policy: a hybrid LQR + RL controller for the standing-balance phase, and a motor-model domain-randomization sweep to characterize the policy’s robustness to actuator mismatch.

Hybrid LQR + RL (negative result)

Motivation: kill the residual wobble in the standing phase. The RL policy is a tanh-MLP - its output near the standing pose is not perfectly smooth in the observation, so small contact-noise perturbations cause action twitches that the policy then has to recover from. A classical linear controller around the standing pose would, in principle, give a smooth

state-feedback that doesn't twitch. Note: this humanoid has no true unforced equilibrium at the standing pose (the body falls under zero control); we therefore linearize about the policy's *empirical operating point* during standing instead of a strict equilibrium. Same idea as gain-scheduling around a quasi-steady operating regime.

Design. The policy's empirical operating point during standing was captured (lowest-velocity standing state seen in a v8 rollout from STANDING init, with the corresponding action taken as ctrl_eq). Discrete one-step Jacobians A, B were computed via `mujoco.mjd_transitionFD` at that point. Discrete ARE solved for the infinite-horizon LQR gain K.

Open-loop linearization: max $\approx 4\%$ /step exponential growth on the worst mode, mild for a discrete system). 8 of 54 eigenvalues lie outside the unit circle; the rest are inside but several sit on it. Closed-loop linearization: max $\approx 1\%$ /step exponential growth - **marginally stable on the unit circle**, not strictly inside. LQR pulls the unstable modes inside but doesn't gain strict asymptotic stability margin. This is already a caveat: the linearization analysis says "borderline stabilizable in the linear regime," not "robustly stabilized."

Three closed-loop modes were tried, all on top of the trained v8 policy (no retraining):

Mode	Description	Wobble (n=16, lower=better)
RL-only	Baseline (just the trained policy)	0.098
Hybrid(switch)	Hard switch to LQR inside standing region	0.166
Hybrid(additive)	smooth gating	0.166
Hybrid(damping)	Velocity-feedback only:	0.090 - 0.108 (depends on)

LQR pulls the linearization's eigenvalues to (marginally) inside the unit circle, but in closed-loop on the nonlinear contact-rich system it produces *more* wobble than the RL policy alone (about 60% worse in switch/additive mode). The velocity-damping variant is at best marginal at small γ (8% improvement at $\gamma = 0.1$, within noise at n=16) and gets worse at larger γ .

Hypothesized diagnosis (asserted; not measured directly here): the trained PPO policy is doing implicit nonlinear feedback well-matched to the contact dynamics. LQR's linear approximation likely produces destabilizing actions in this regime because (a) the DARE gains are high (Frobenius norm ≈ 10 at the default Q/R), so the controller amplifies state noise the linearization treats as smooth dynamics, and (b) the actual operating point during standing varies slightly from the chosen linearization point, so LQR persistently pushes against poses the policy prefers. A direct measurement (e.g., comparing $\gamma = 0.1$ vs $\gamma = 0.01$ at contact-rich states) would confirm or refute this; it wasn't done here.

This is itself a controls result: the textbook answer (LQR around the standing operating point) underperforms here, and the reason is the system's contact-rich nonlinearity, which is exactly what makes humanoid balance non-trivial in the first place.

Motor-model domain randomization (positive result)

Motivation: the trained policy uses fixed dm_control actuator dynamics. A real robot’s motors have unit-to-unit variation in gain, response lag, and torque limits. How robust is the trained policy to motor-model mismatch?

Setup. A MotorDRHumanoidGetUp env wrapper applies a per-joint transformation to the policy’s commanded action before it reaches the simulator:

where γ_i are per-joint and sampled once per episode from uniform ranges. The policy does *not* observe these parameters - it has to be robust to motor variations it cannot see.

Severity levels:

Level	gain	(sec)
nominal		
mild		ms
strong		ms
extreme		ms

v8 result (no DR in training):

Severity	Success rate	Mean wobble
nominal	100%	0.094
mild	100%	0.116
strong	100%	0.166
extreme	62%	0.236

 Motor-DR robustness
Motor-DR robustness

The trained v8 policy keeps a 100% success rate up to mild motor variation, while its wobble grows substantially (0.094 → 0.166, +77%) - the *task* still succeeds but the standing quality degrades meaningfully on the way. At strong (extreme), success drops by 38 percentage points in a single severity step (100% → 62%) and wobble grows another 42%; side-init is the worst case at 40% success. Two caveats on the extreme tier specifically: - some mild samples produce motors physically too weak to lift the body (e.g., 0.05 sec per joint on a leg), so failure there conflates policy capability with physical feasibility - we don’t attempt to separate them here; - 100 episodes per severity is small; success rate has a wide CI at every cell. The shape of the curve is meaningful, the per-cell numbers shouldn’t be read as precise.

The conclusion is more limited than “robust”: v8 *tolerates* motor mismatch up to mild at task level while quality degrades, and breaks beyond that. A policy trained explicitly with DR (v9) was attempted but was undertrained at our compute budget and

underperformed v8 across all severities.

6. Limitations & Future Work

Where the policy fails

The policy reaches stable upright but doesn't *maintain* it indefinitely. Observed pattern: stands → loses balance (often via tippy-toe foot) → falls to knees or fully supine → re-executes the getup → stands again. The cycle gets reward (head_bonus fires throughout) but isn't true sustained balance.

Specific failure mechanisms: - **Tippy-toe foot.** The stock reward doesn't penalize foot orientation in any of its terms. - **No stability bonus.** Reward fires equally for transient and sustained standing. No specific incentive to hold rather than fall-and-recover. - **Self-reinforcing fall-recover loop.** Because the policy is good at getup, falling is "cheap" in reward - the dense head_bonus + recovery motion still pay out.

The v8 redesign added foot-flat + stillness penalty (both gated on head_z), which improved foot quality (foot_flat ~2.3× higher at matched compute - v8@200M vs v7@200M peer; the 7.5× headline in §4 is v8-long@413M vs v7@530M, which mixes the redesign with additional training) and cut eval std 32% lower (143 → 97). The remaining gap is the fall-recover oscillation that still happens for a small fraction of episodes.

What would close the remaining gap, in order of leverage per effort

1. **Termination on fallen state.** If head_z < 0.3 for >50 consecutive steps, end the episode (no penalty, just truncate). Removes the fall-recover loop directly.
 2. **Learning-rate decay** in the last 30% of training. v8-long oscillated between high and partial-collapse evals because the constant lr kept the policy moving. LR decay would lock in a good basin.
 3. **State-init curriculum decay** (mix probability shrinks toward 0% standing as the policy improves). We used a fixed uniform mix; the standard recipe in getup literature anneals it.
 4. **Short reference for kneel→stand:** a 50-frame interpolated reference trajectory used as an additional tracking reward only in the transition zone. Most reliable fix, most engineering cost.
 5. **Drag-assist:** a virtual upward force on the head, decayed over training. Held in reserve - our recipe didn't need it for getup.
-

7. Engineering Decisions

- **Variable-isolation discipline** - one intentional change per iteration after the v3 two-change run; that's exactly what surfaced the supine-quat bug as soon as the next single change (v4) was made.
- **Stock-baseline validation** before adding any subclass caught a Modal image / SSL bug and the warp/jax backend mismatch up-front, before either could waste iteration time.

- **Per-eval checkpointing** via brax PPO’s `policy_params_fn` saved the deliverable: v8-long oscillated, `final.pkl` landed on a trough, and the best policy was at an intermediate eval. Without per-eval snapshots, the run would have been a wash.
-

8. Code

- `src/controls_playground/env.py` - HumanoidGetUp (init mix, reward extension)
- `src/controls_playground/scripts/train.py` - brax PPO + per-eval checkpointing
- `src/controls_playground/policy.py` - checkpoint loader (auto-detects hidden sizes from the params blob)
- `src/controls_playground/scripts/viz.py` - viewer / mp4 / per-episode clip CLIs
- `src/controls_playground/scripts/probe.py` - out-of-distribution init probe
- `src/controls_playground/scripts/mpc.py` - MPPI baseline (plan / replay)
- `src/controls_playground/scripts/landscape.py` - empirical state-landscape plot
- `src/controls_playground/scripts/analysis.py` - local linearization probe
- `modal_dispatch.py` - H100 dispatcher
- `configs/{env,train,mpc}.yaml` - all magic numbers live here.

9. Tools and external code

Libraries (used unmodified): brax PPO (Google), mujoco_playground / dm_control_suite humanoid (DeepMind), MuJoCo / MJX, Modal for H100 dispatch, wandb for logging.

Reference material: HumanUP (Dong et al., *Learning Getting-Up Policies for Real-World Humanoid Robots*, RSS 2025) for the state-init-mix and exponential-reward ideas. That work targets a different system (Unitree G1 with co-optimized motor parameters and Isaac Gym) and a heavier two-stage recipe (drag-assist, Stage I discovery → Stage II tracking). I adopted only the two ideas above and validated each in isolation; the rest of the recipe was out of scope (see §1b).

AI assistance: I worked with an AI coding assistant (Claude) on boilerplate (Modal image, plot scripts, argparse, YAML loader, package restructure), mechanical debugging (e.g., the brax PPO `num_envs × unroll_length` divisibility constraint that produced NaN in v1), and the per-episode clip renderer. Engineering decisions, diagnoses, and the final calls (5-way init mix, v8 reward redesign, ship-the-413M-peak) were mine.

Appendix A: PPO formulation

State (joint angles, joint velocities, head height, end-effector positions in torso frame, torso vertical, CoM velocity). Action (joint torque commands).

Init distribution.

with $\theta = 0.01$ rad and $\phi = 0.01$ a snapshot from a partial-getup rollout.

First line is the stock dm-control multiplicative balance reward; lines 2-3 are added here.

; is head , m,

; are the feet's vertical alignment (1 = sole flat); , ,

, m. The indicator gate ensures the new terms only fire in the standing region - they shape stability, not getup.

Appendix B: MPPI stage cost

with $\alpha = 0.01$ and $\beta = 0.001$ in the late-phase velocity-damping term. Parameters from `configs/mpc.yaml`: `samples`, `horizon`, `smoothing length`, `temperature`, `substeps`, `steps`.

Appendix C: Empirical state landscape

The 54-D nonlinear contact dynamics doesn't admit an analytic "feasible region" (the way a 2-D LTI system does), so we project onto the two task-relevant scalars

- head height and torso-up alignment - and overlay: - a heatmap of the per-step reward on a synthetic grid over that slice, - trajectories of the trained policy from each init type.

The standing target corner is .

Landscape
Landscape